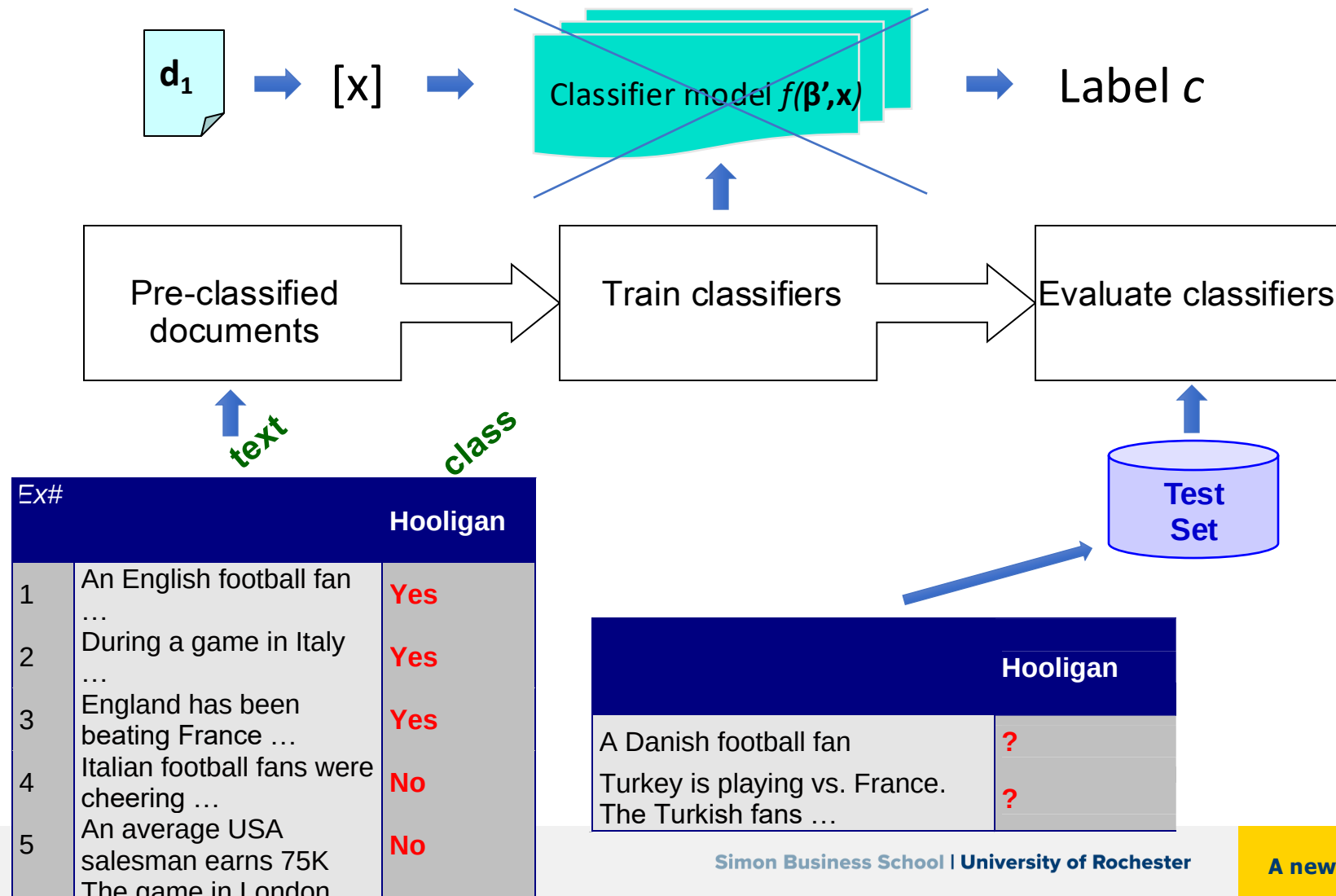


S7: Neural Networks

Review

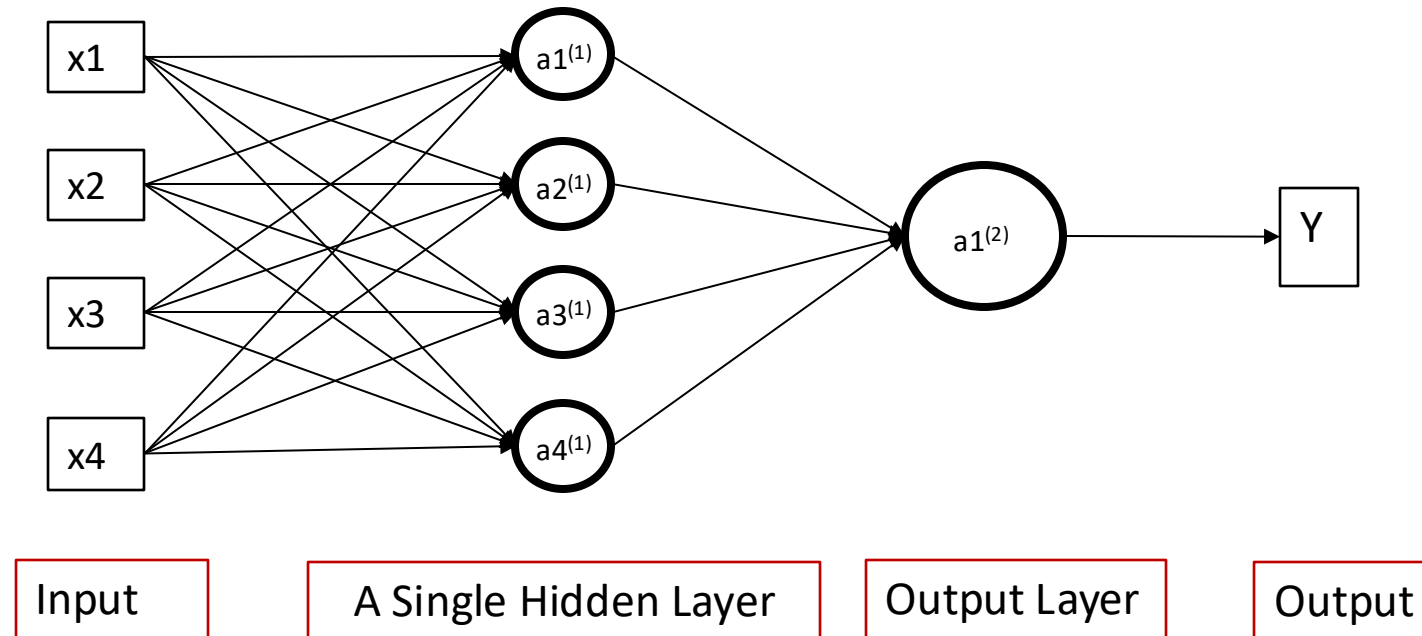


Candidate classification models

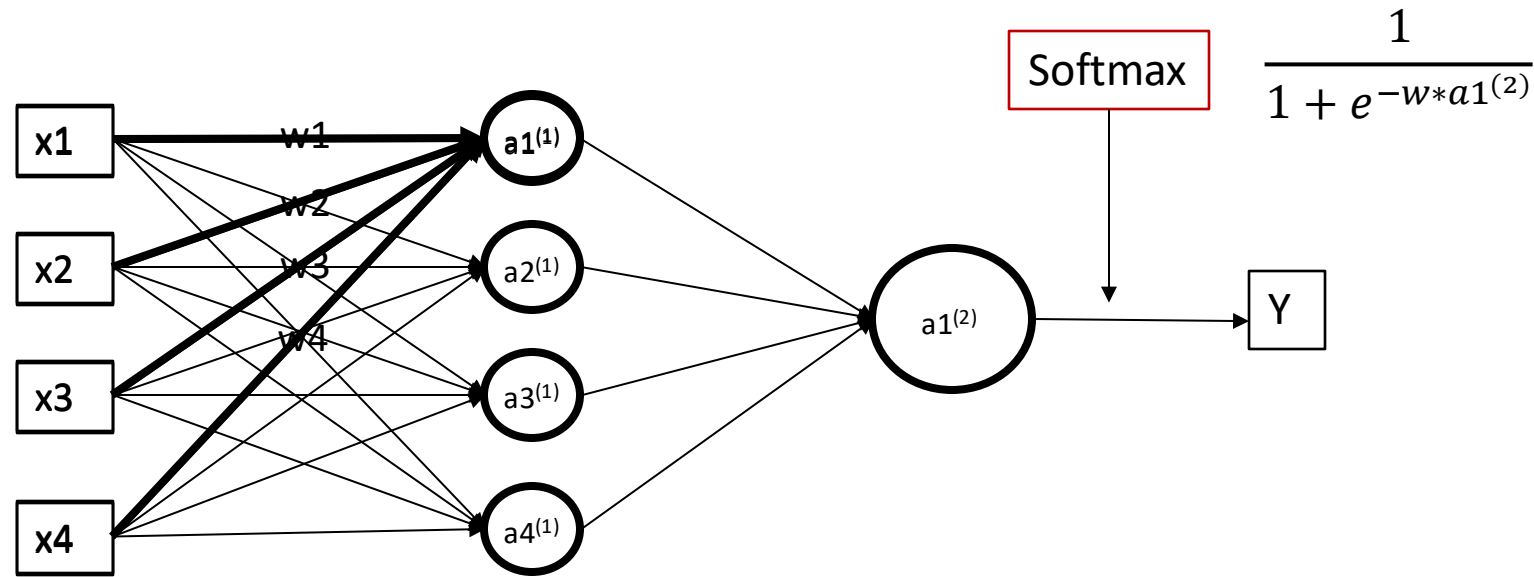
- **Naïve Bayes**
 - **Logit Model**
 - **Support Vector Machine**
 - **Decision Tree(Forest)**
 - Neural Networks
 - Deep Neural Networks
 - ...
- classic, works well (70% accuracy),
computationally savvy (CPU)
- advanced, works well (80% accuracy),
computationally savvy (CPU)
- frontier, works very well (90+% accuracy),
computationally heavy (GPU)
- Note: using the **same** training set and testing set for comparing the performance of alternative models

Artificial Neural Network

- A model inspired by the biological nervous systems, such as the human brain's information processing mechanism
- **Ensemble** model: similar to random forests



Artificial Neural Network



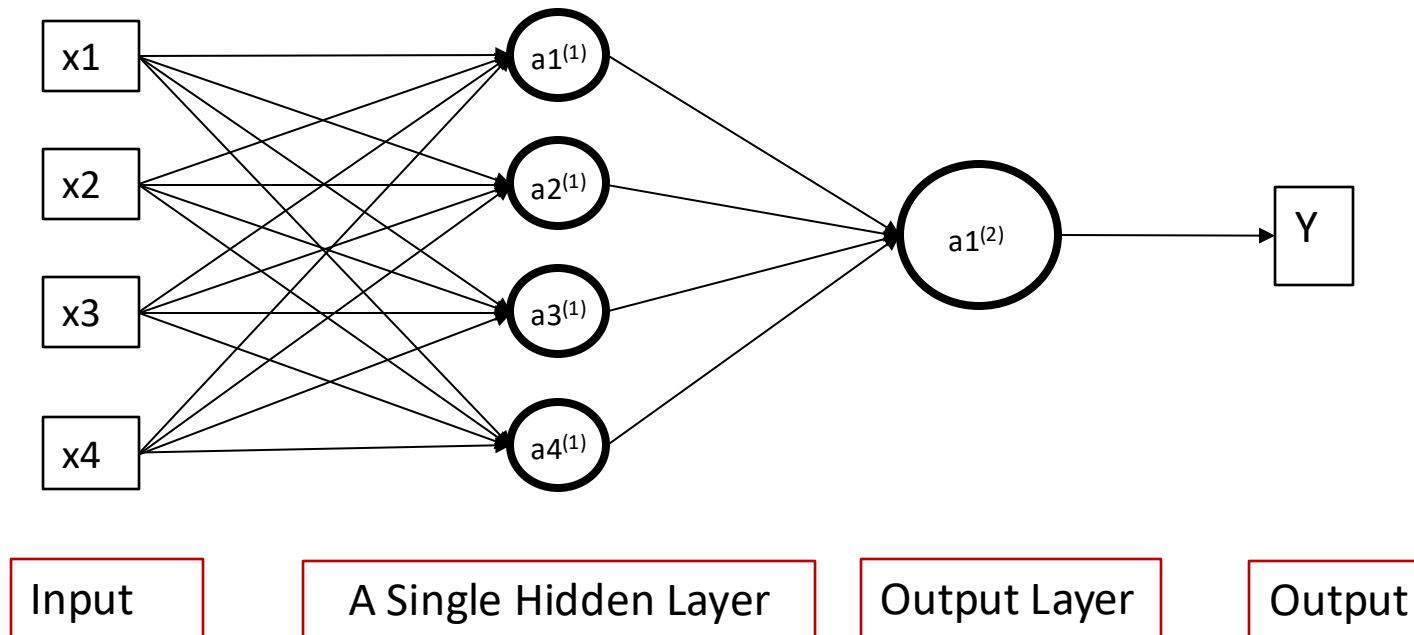
- $a_1^{(1)} = f(w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + w_4 * x_4)$
- $f()$ is activation function: **Rectified Linear Unit (ReLU)** or **Sigmoid**
 - e.g., ReLU $\max(0, x)$
 - $a_1^{(1)} = \max(0, w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + w_4 * x_4)$
- $a_1^{(2)} = f(w_1' * a_1^{(1)} + w_2' * a_2^{(1)} + w_3' * a_3^{(1)} + w_4' * a_4^{(1)})$
- Number of parameters?
- What if we have **0** hidden layer?

Compare with Logit model

- Pro?
- Con?

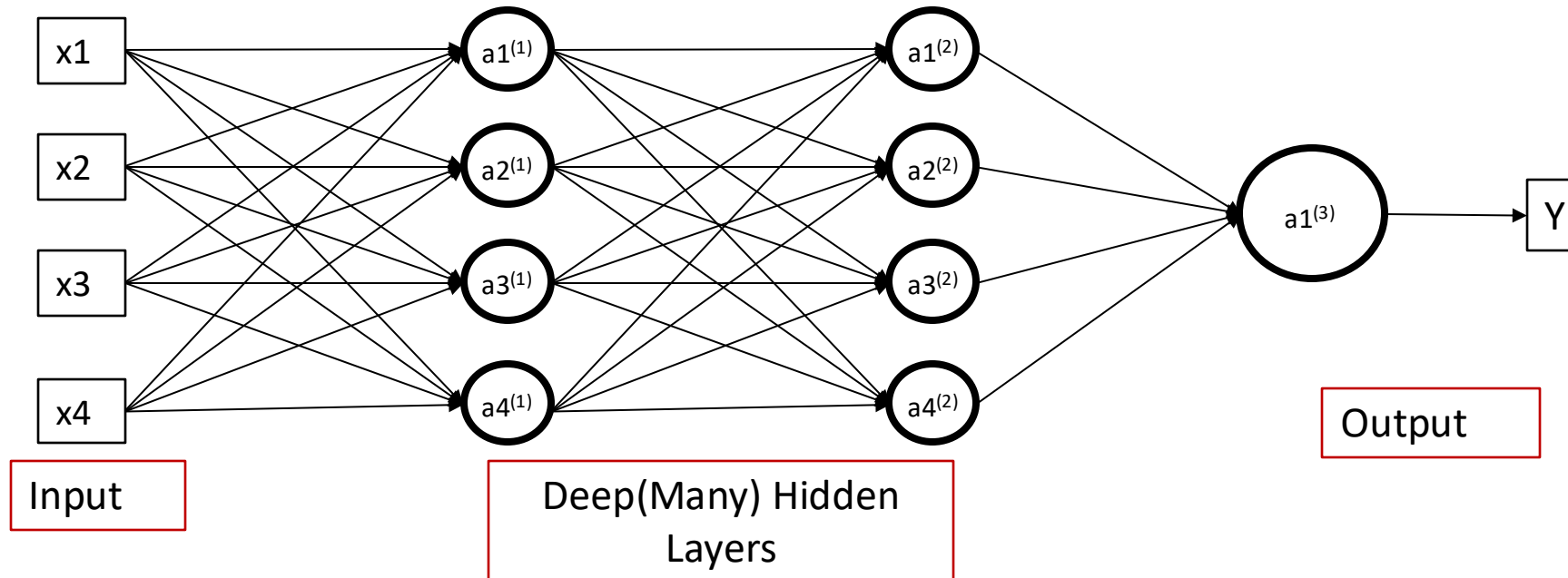
How to further improve the model?

- Option 1: increase the neurons at the same layer
 - How many more parameters needed if add 4 more neurons?
 - What if the length of input is 1,000?



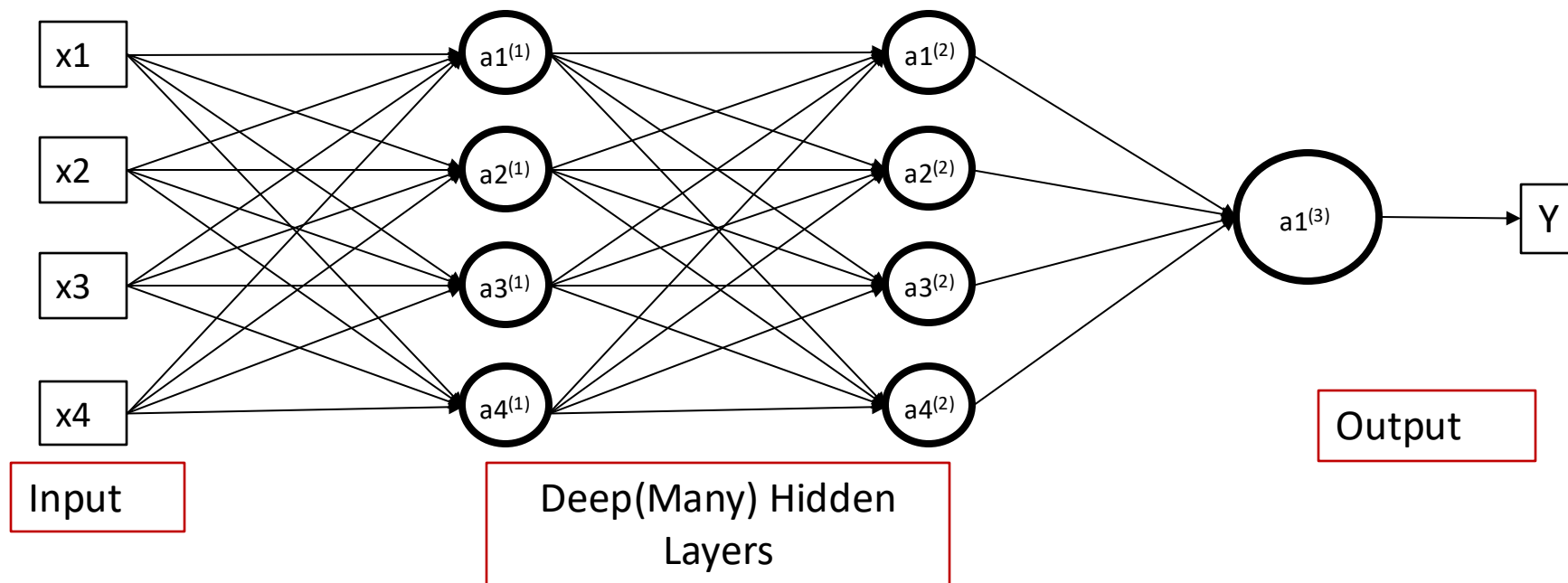
How to further improve the model?

- Option 2: add neurons in a second and new layer
 - $a1^{(2)} = f(w1' * a1^{(1)} + w2' * a2^{(1)} + w3' * a3^{(1)} + w4' * a4^{(1)})$
 - $f()$ is activation function: **Rectified Linear Unit (ReLU)** or Sigmoid
 - How many more parameters needed if add four more neurons?
 - What if the length of input is 1,000?



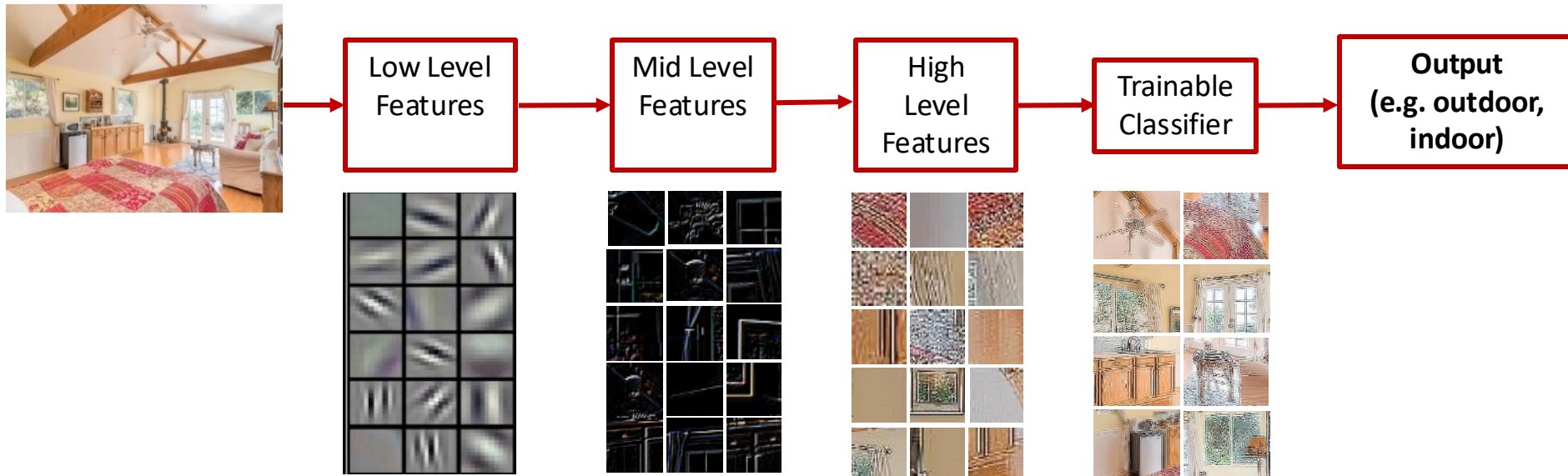
Deep Neural Network

- What if I have the 3rd layer? 4th layer?
- Many layers? Very deep neural networks
 - How many more parameters needed if I have 4 layers with 4 neurons each?
 - What about 1 layer with 4*4 neurons?

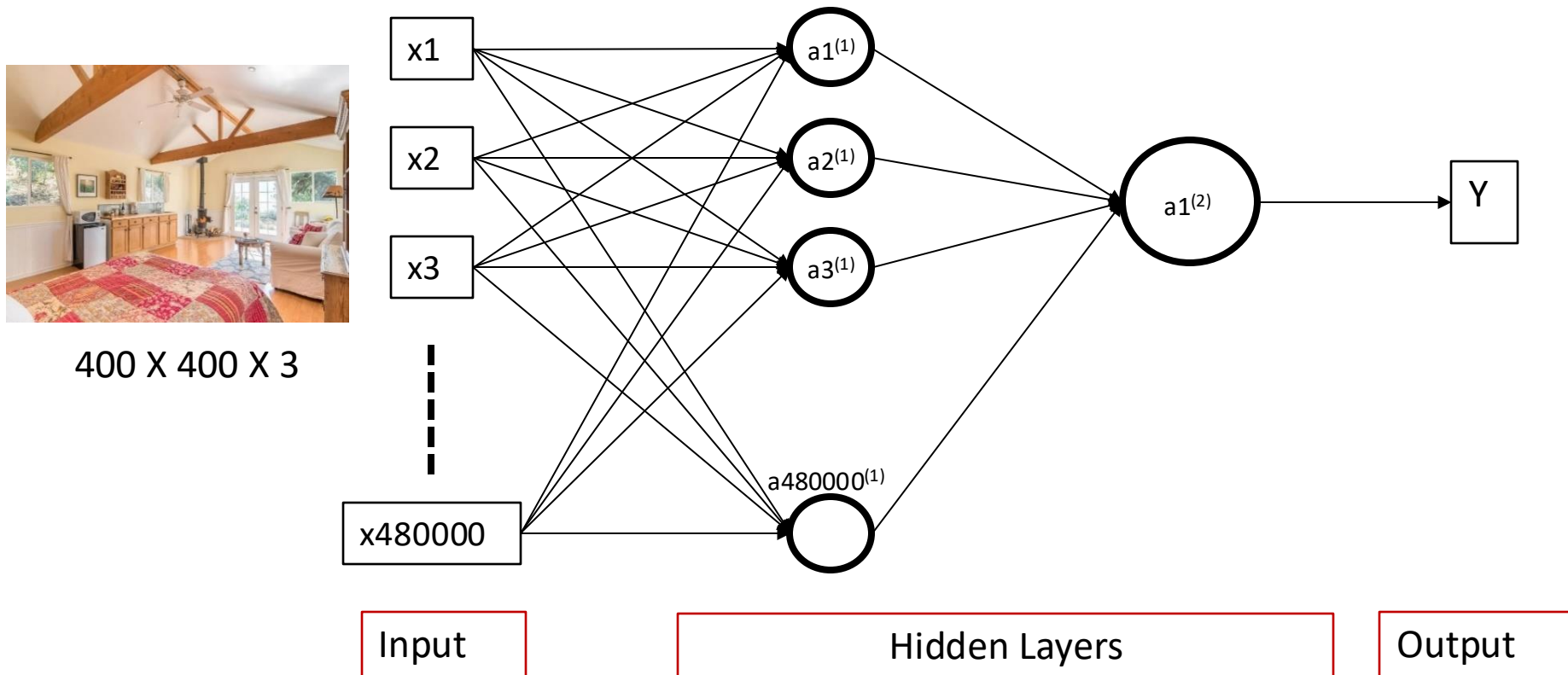


What do those layers mean?

- Deep learning has an **inbuilt automatic multi-stage feature learning process** that learns rich hierarchical (non-linear) representations (i.e., features).



If the input is long

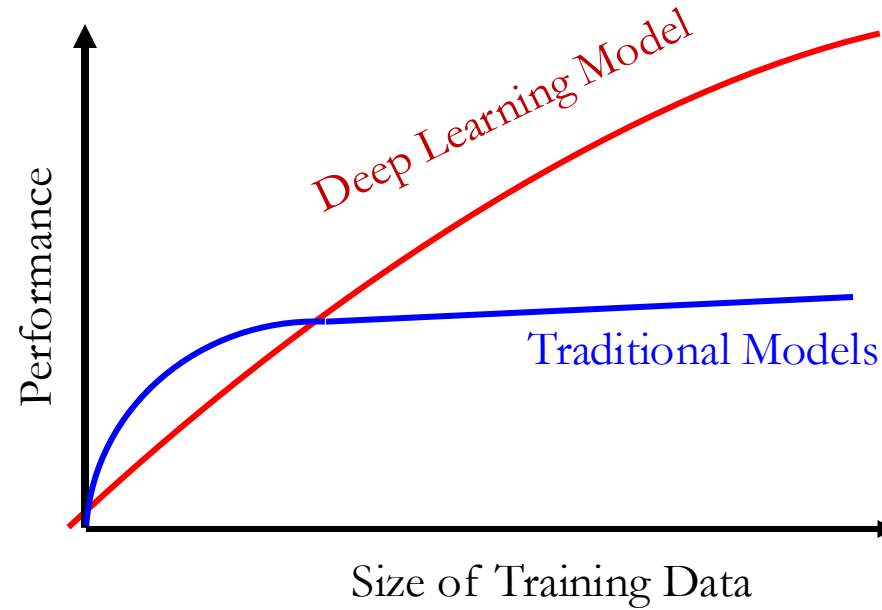


Number of parameters

- $480000 \times 480000 + 480000 + 1 = \text{approximately } 230 \text{ Billion!!!}$
- $480000 \times 1000 + 1000 + 1 = \text{approximately } 480 \text{ million!!!}$
- How many images I need?

Performance of deep learning

- When data size is small?
- When data size is large?



Neural net architectures

- **Output:** Binary, Multinomial, Continuous, Count
- **Input:** fixed-size vector
- **Architecture:** increasing number of layers (depth) makes neural net more complex
 - **Pro:** approximate more functions
 - **Con:** learn more parameters, need more training data.
 - **Best: Ad-hoc**

Python: Neural networks

```
## Neural Network and Deep Learning

from sklearn.neural_network import MLPClassifier

DLmodel=MLPClassifier(solver='lbfgs',hidden_layer_sizes=(3,2),random_state=1)

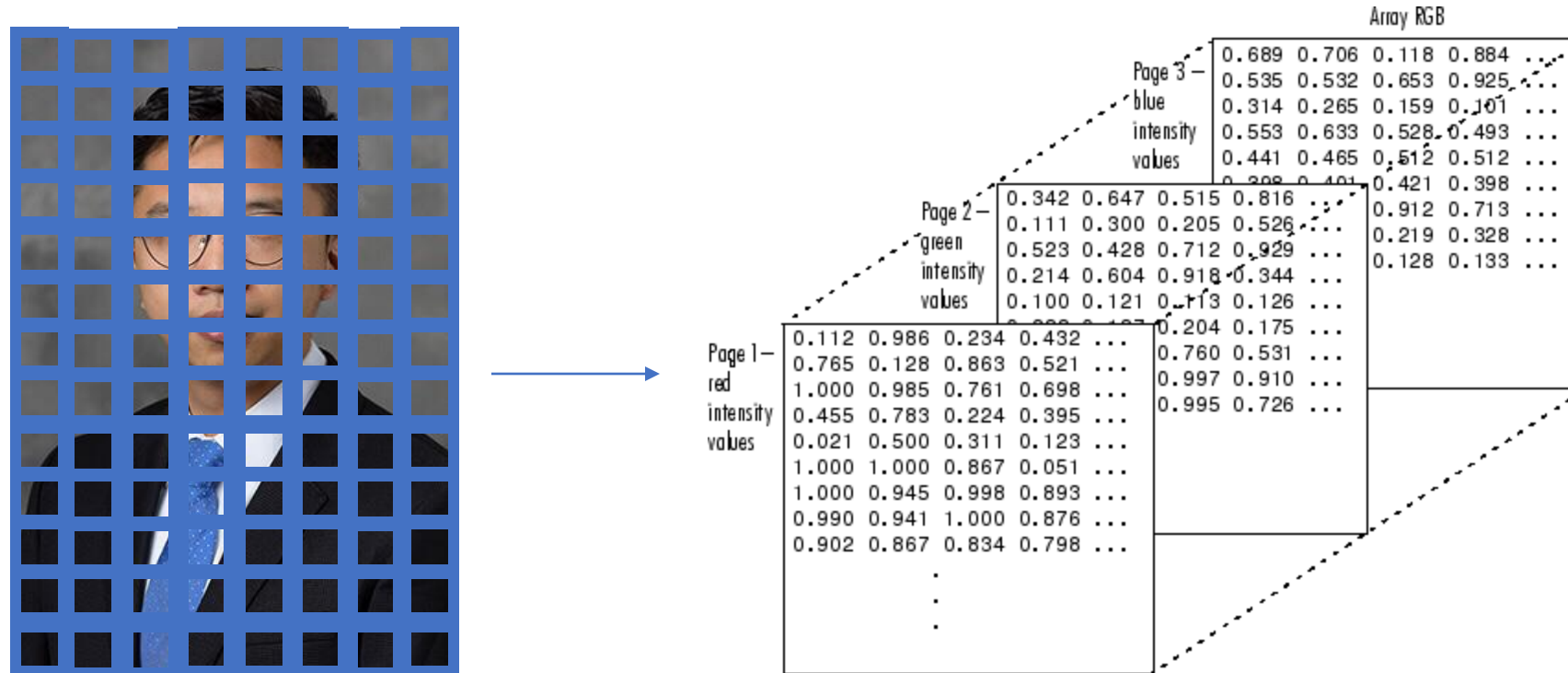
# training
DLmodel.fit(training_x, training_c)
y_pred_DL=DLmodel.predict(testing_x)

# evaluation
acc_DL = accuracy_score(testing_c, y_pred_DL)
print("DL model Accuracy: {:.2f}%".format(acc_DL*100))
```

Mini Group Project 3a

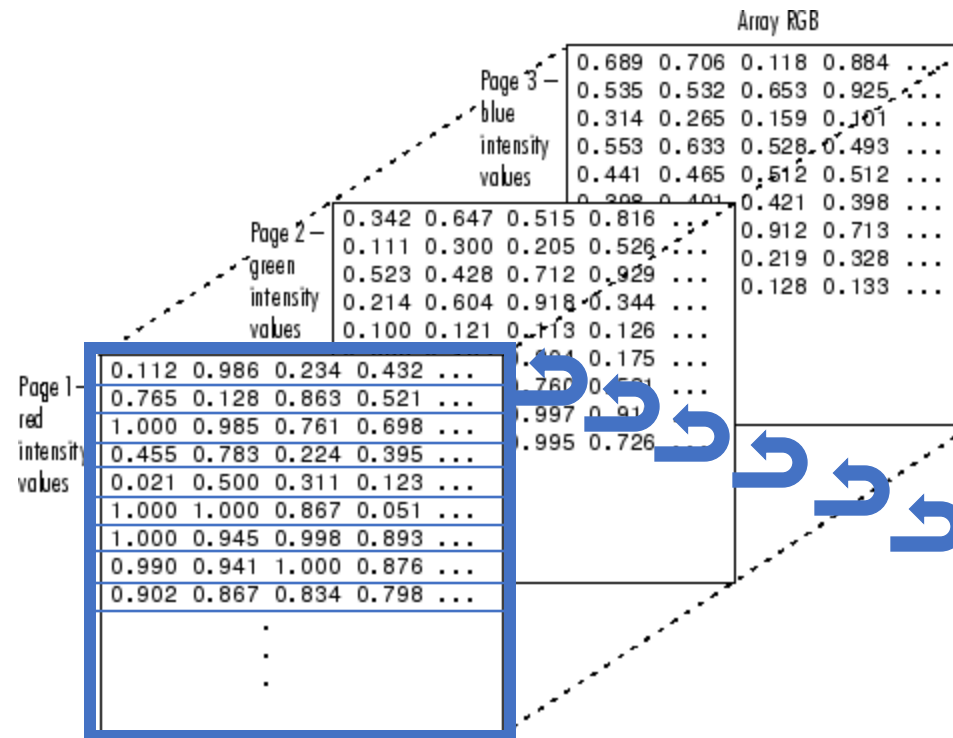
- Based MGP2b, lets further include some neural network models for comparison.
- 1. Model 1 includes 1 layers of 4 neurons in total.
- 2. Model 2 includes 2 layers of 2 neurons in each layer.
- 3. Model 3 includes 3 layers, with 2, 1, and 1 neurons in sequence.
- Using the same representations as is in 2b, report those three models' performance and discuss which model performs the best according to the accuracy rate. Also compare with the previous models' performance
- Create a 3 by 3 table, table reporting the accuracy rates of the 3 models under the 3 types of representations

Object detection in image



Representation

- Array representation: 3D
 - Array: $I[:, :, :]$
- Keep grayscale only: 2D
- Vectorization: 1D



[0.40392157, 0.39607843, 0.4, ..., 1., 1., 1.]

Python: image representation

```
import numpy
from PIL import Image #after pillow is installed
from pylab import *
import matplotlib.pyplot as plt

##image manipulation
im = Image.open('zjy.png')
im_rs = im.resize((100,100))
im_rotate = im.rotate(180)

plt.imshow(im)
imshow(im_rs)
imshow(im_rotate)

##representation
im_m = array(im)
im_grey_m = array(im.convert('L'))
im_v = im_m.flatten()
```

Representation fits classic classification models

- $[x]$ should be a vector

Python: digit recognition

```
# Import image data
from keras.datasets import mnist
import matplotlib.pyplot as plt
import numpy as np

# Split the data between train and test
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print(x_test.shape)

# Flatten the images
x_train_v=x_train.reshape((x_train.shape[0],x_train.shape[1]*x_train.shape[2]))
x_test_v=x_test.reshape((x_test.shape[0],x_test.shape[1]*x_test.shape[2]))
```

Python: digit recognition

```
## Train a RF model
from sklearn.ensemble import RandomForestClassifier
model1 = RandomForestClassifier(n_estimators=150)
model1.fit(x_train_v, y_train)
y_pred_1 = model1.predict(x_test_v)

# evaluation
from sklearn.metrics import accuracy_score
acc_1 = accuracy_score(y_test, y_pred_1)
print("Random Forest model Accuracy: {:.2f}%".format(acc_1*100))

from sklearn.metrics import confusion_matrix
cm1 = confusion_matrix(y_test, y_pred_1)
print(cm1)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_1))
```